

Ekstrakcija informacij in poizvedovanje po vsebini z uporabo vektorskih modelov

Projektna naloga pri predmetu Iskanje in ekstrakcija podatkov s spleta

V drugi projektni nalogi smo raziskovali ekstrakcijo pomembnih informacij iz pridobljenih HTML dokumentov prve naloge, njihovo shranjevanje v vektorsko podatkovno bazo in poizvedovanje po vsebini s pomočjo semantičnega iskanja. Uporabili smo pristope ekstrakcije vsebine spletne strani z XPath izrazi, regularnimi izrazi ter vektorsko iskanje z različnimi modeli in metrikami za poizvedovanje.

1. Ekstrakcija spletnih strani

Za namene ekstrakcije HTML vsebine zajetih spletnih strani, smo pripravili cevovo, ki izvaja preprocesiranje vsebine, pripravljene za shranjevanje v vektorsko bazo z izračunanimi vektorskimi predstavitvami (embeddings).

Najprej preko API klica pridobimo vse strani, primerne za obdelavo – torej tiste, ki nimajo prazne vsebine. Vse strani, primerne za obdelavo, ki jih je bilo v našem primeru nekaj manj kot 6000 nato obdelamo po postopku, opisanem v nadaljevanju.

1.1 Čiščenje HTML strukture

Odstranimo nepomembne HTML elemente in v grobem izoliramo glavno vsebino. Naša implementacija z uporabo knjižnice BeautifulSoup identificira osrednjo vsebino strani preko `id="content"`, odstrani pa neželene statične elemente, kot so:

- Vnosna podlga (`<form>`),
- meniji (`<nav>`, `<aside>`),
- oglasi in skripte (`<script>`, `<iframe>`, `<style>`),
- noge strani (`<footer>`),

1.2 Ekstrakcija podatkov

V nadaljevanju pridobimo želeno vsebino iz prečiščene HTML strukture s pristopoma XPath in Regex.

1.2.1 XPath ekstrakcija vsebine

Za strukturirane elemente, kot so naslovi (`<h1>`), kategorije (`<li class="categories">`) ali datum (`<time>`), smo uporabili XPath izraze. Primeri uporabljenih izrazov:

XPath izraz	Namen
<code>//h1[@class="current"]/text()</code>	Pridobivanje naslova članka
<code>//li[contains(@class, "categories")]/a/text()</code>	Pridobivanje kategorije članka

//div[@class="besediloNovice"]//text()	Pridobivanje vsebine članka
//div[contains(@class, "post")]	Pridobivanje vseh uporabniških komentarjev
./h4/a/text()	Avtor komentarja
./time/@datetime	Čas objave komentarja
./div[@class="content"]//text()	Besedilo komentarja
./div[@class="content"]/*[not(self::p[@class="user-quoted"] or self::blockquote)]/text()	Izločanje citiranih delov iz komentarjev

1.2.2 Regex ekstrakcija vsebine

Za vsebino, ki ni bila zanesljivo dostopna z XPath, smo uporabili regularne izraze. To je bilo potrebno za pridobivanje avtorjev člankov in datuma objave:

Regularni izraz	Namen
\s*(.*?)\s*	Pridobivanje avtorja članka
<time datetime="([^\"]+)"	Pridobivanje ISO datuma

1.3 Strategija razdeljevanja besedila na segmente

Delitev na segmente smo implementirali z različnimi pristopi, ki so se izkazali za najbolj smiselni način delitve glede na našo vsebino. Ta vsebuje pridobljene strani s spletnega mesta Slo-Tech, ki vsebuje krajše novice, forume in komentarje pod novicami. Na podlagi tega smo lahko določili osnovno delitev na segmente, ki smo jo že v postopku ekstrakcije delili glede na vrsto vsebine. Tako smo pridobili tri glavne vrste segmentov:

- Naslovi – predstavljajo samostojne segmente, ki služijo kot osnovna identifikacija vsebine
- Odstavki člankov – vsebujejo glavni del besedila članka
- Komentarji – predstavljajo uporabniško generirano vsebino, ki se navezuje na članek

Glavno besedilo člankov nato še naprej razdelimo po odstavkih in jih analiziramo glede na dolžino. Kratke odstavke združujemo za ohranjanje konteksta. Če je trenutni segment krajši od minimalne dolžine (50 znakov), ga združimo z naslednjim odstavkom. Če bi dodajanje odstavka povzročilo prekoračitev maksimalne dolžine (500 znakov), pa zaključimo trenutni segment in začnemo novega. Ta pristop uravnoteži potrebo po smiselnih enotah in optimalni dolžini za vektorske predstavitve.

Vsak ustvarjen segment ima metapodatke, ki so v nadaljevanju uporabni za prikazovanje rezultatov poizvedb. Med te metapodatke spadajo naslov članka, tip segmenta (naslov, odstavek ali komentar) in kategorija vsebine na spletni strani Slo-Tech (npr. Znanost in tehnologija, zasebnost, tožbe, kriptovalute itd.). Ti podatki omogočajo boljše razumevanje konteksta pri rezultatih poizvedbe.

Prednost naše implementirane strategije je, da smiselno delimo vsebino spletne strani glede na ločeni tip vsebine in s pomočjo metapodatkov zagotovimo ves potreben kontekst. Na ta način

ne diskriminiramo iskanja po enem tipu vsebine, saj je za uporabnika lahko zanimivo pridobiti vse vrste vsebin za iskano temo (vključeni so tudi komentarji). Trenuten pristop je možno še dodatno razširiti s tem, da pri iskanju v naslednjem koraku nudimo filtriranje glede na tip vsebine. Ta način se nam je tako zdel najbolj smiselni in nudi najširše možno poizvedovanje, z možnostjo mnogih razširitev.

Kljub prednostim ima naša strategija tudi nekaj pomanjkljivosti. Fiksne omejitve velikosti segmentov morda niso optimalne za delitev vsebine člankov. Poleg tega se pri našem pristopu zanašamo na delitev besedila na odstavke, ki pa morda v določenih primerih ni prisotno.

Naša strategija temelji predvsem na strukturi in ne na pomenski analizi besedila. Ne upošteva semantičnih povezav med odstavki, temveč se zanaša predvsem na vidne prelome v besedilu, kar je lahko v nekaterih primerih neoptimalno.

2. Shranjevanje v vektorsko podatkovno bazo

Uporabili smo razširjeno PostgreSQL podatkovno bazo s podporo za pgvector. Dodali smo novo tabelo `page_segment` s poljem `embedding` tipa `vector(768)`, kjer hranimo vektorske predstavitve segmentov in dodali indeks `IVFFlat`. Za izboljšano iskanje in kontekst smo v tabelo dodali še dodatna polja za metapodatke, ki smo jih našli pri razdelitvi strani.

2.1 Izračun vektorskih predstavitev

Preizkusili smo dva modela za generiranje vektorskih predstavitev segmentov in sicer LaBSE in SloBERTa. Ključnega pomena je bilo, da oba podpirata slovenski jezik, pri čemer je model SloBERTa treniran posebej za slovenščino. Preizkusili smo delovanje obeh in se na koncu odločili za primarno uporabo modela LaBSE, saj je zagotovil nekoliko boljše rezultate pri poizvedbah. Razlog za tem je najverjetneje v dejstvu, da naša vsebina predstavlja kar nekaj angleških in strokovnih izrazov, ki morda niso tako dobro podprti pri drugemu modelu. Kljub temu smo v končni implementaciji ohranili možnost predprocesiranja vsebine z modelom SloBERTa, pri čemer moramo izbrati ustrezno računanje vektorskih predstavitev z izbranim modelom tudi pri poizvedovanju v uporabniškem vmesniku.

3. Poizvedovanje

Za preprosto izvajanje poizvedb smo implementirali program `demo.py`, ki vključuje spletni uporabniški vmesnik z izbiro različnih iskalnih parametrov. Preden uporabnik pošlje poizvedbo, lahko izbira enega izmed treh možnih pristopov iskanja v vektorski bazi:

- cosine (kosinusna podobnost),
- L1 (Manhattanska razdalja) in
- inner product

Vsi trije pristopi so se pri našem testiranju izkazali za učinkovite, z manjšimi, nepomembnimi odstopanji. Poleg tega vmesnik omogoča izbor modela (LaBSE/SloBERTa), ki se mora ujemati z načinom računanja vektorskih predstavitev v predprocesiranju, uporabnik pa lahko izbere tudi želeno število rezultatov (npr. top-5 ali top-10).

Na sliki 1 lahko vidimo primer poizvedbe za besedo "Mozilla" in 5 zadetkov poizvedbe.

Slika 1 Rezultat poizvedbe besede "Mozilla"

Vector Search Demo

Mozilla					
-- Choose a predefined query --	Cosine Similarity	LaBSE	5	Search	

Search Results

Similarity: 0.8329

Segment Type: title

Title: Mozilla Firefox

URL: <https://slo-tech.com/forum/t619719/199>

Mozilla Firefox

Similarity: 0.6843

Segment Type: title

Title: Mozilla Firebird v slovenskem jeziku

URL: <https://slo-tech.com/novice/t136772>

Mozilla Firebird v slovenskem jeziku

Similarity: 0.6429

Segment Type: comment

Title: Mozilla predlaga nov način varovanja zasebnosti na internetu

URL: <https://slo-tech.com/novice/t451946>

Mozilla nas trolla.

Similarity: 0.6358

Segment Type: title

Title: mozilla firefox 4.0

URL: <https://slo-tech.com/forum/t505663>

mozilla firefox 4.0

Similarity: 0.614

Segment Type: comment

Title: Pametni telefoni: LG ima dovolj

URL: <https://slo-tech.com/novice/t779213/0>

motorola

4. Omejitve sistema

Naš trenutni sistem za iskanje dokumentov se sooča z več omejitvami, ki vplivajo na njegovo učinkovitost in uporabnost. Sistem uporablja fiksne velikostne omejitve za razdelitev besedila (med 50 in 500 znakov), kar pogosto ne upošteva pomenskih enot besedila. To lahko vodi do neprimerne razreza idej ter izgube konteksta.

Sistem ne podpira iskanja oz filtriranja različnih tipov vsebine, saj enakovredno obdeluje novičarske članke, komentarje in naslove člankov, kar ni optimalno, saj imajo ti različni tipi besedil čisto drugačne pomene in uporabnost.

Sistem prav tako ne upošteva časovne komponente pri razvrščanju rezultatov iskanja, kar pomeni, da starejše in potencialno manj relevantne vsebine dobijo enako prioriteto kot najnovejše informacije.

Vse to so možne izboljšave, ki so direktno vezane na specifično vsebino naših zajetih strani. Kljub temu sistem uspešno zagotavlja glavno funkcionalnost, ki je iskanje po ključnih besedah, omogoča pa še mnoge razširitve.